PrismGS: Coordinated Training and Compression for Pareto-Optimal 4D Gaussian Streaming

Anonymous Author(s)

Abstract

4D Gaussian Splatting (4DGS) has recently become a leading representation for photorealistic dynamic scene rendering. However, streaming 4DGS in real-time presents a fundamental trade-off between visual fidelity and data volume. Existing methods suffer from a decoupled training-compression pipeline and employ coarse-grained, uniform bitrate allocation across Gaussian attributes during compression. This leads to delivery inefficiency, yielding suboptimal rate-distortion performance. To bridge this gap, we propose PrismGS, a coordinated training and compression framework for 4D Gaussian streaming that realizes compression-aware dynamic scene reconstruction and attribute-level fine-grained bitrate allocation. First, we propose keyframe regularization constraining Gaussian scale and anisotropy during initialization to mitigate error propagation in dynamic training and quantization-induced artifacts in later compression stage. We then adopt a residual-based dynamic training pipeline to ensure temporal coherence and high compressibility. Finally, we propose a dynamic programming algorithm to identify the Pareto-optimal bitrate allocation by leveraging the heterogeneous sensitivity of Gaussian attributes to compression. In addition, we introduce a parallel decoding scheme in client side, significantly reducing decoding latency. Extensive experiments demonstrate that PrismGS outperforms existing methods by up to 1.3 dB in PSNR, 30% size reduction, 15% faster rendering speed and 4 times faster decoding speed in various datasets.

CCS Concepts

• **Computing methodologies** → **Volumetric models**; • **Information systems** → **Multimedia streaming**.

Keywords

4D Gaussian Splatting, Dynamic Scene Reconstruction, Volumetric Video Compression, Rate-Distortion Optimization

1 Introduction

3D Gaussian Splatting (3DGS) [15] has revolutionized novel view synthesis by enabling photorealistic rendering at interactive frame rates through explicit 3D Gaussian representations. Beyond static scenes, 4D Gaussian Splatting (4DGS) extends 3DGS to dynamic settings by adding time variants to 3DGS representations and has attracted growing attention. 4DGS methods typically exploit temporal coherence across frames to reduce computational cost, achieving efficient training and rendering by propagating Gaussian representations over time [19, 37]. These advancements enable the reconstruction and rendering of time-varying scenes for a variety of applications, such as immersive telepresence and operation [20, 25], autonomous driving simulation [8, 27, 46], and virtual reality (VR) [14, 16, 17].

A common paradigm in 4DGS decomposes a sequence of input views or images into a keyframe and a set of subsequent dynamic frames [7, 29]. The keyframe is reconstructed using standard 3DGS optimization, while subsequent frames are obtained by predicting and refining inter-frame variations of Gaussian attributes. By avoiding per-frame retraining from scratch, this type of training strategy significantly improves efficiency while maintaining temporal consistency. Recent works further incorporate lightweight motion prediction and compact dynamic scene representations [11, 33, 44] to further improve efficiency and visual quality, forming a practical backbone for 4DGS training.

Despite these advances, we observe that two critical aspects of 4DGS pipelines remain unexplored. First, existing methods still mostly inherit the original 3DGS for keyframe reconstruction, which solely optimizes static visual fidelity. However, in dynamic settings, the keyframe serves as a fundamental anchor for motion prediction, temporal propagation, and subsequent compression. We find that keyframe representations optimized only for static reconstruction may exhibit large Gaussian scale and extreme anisotropy, which can destabilize dynamic finetuning and amplify quantization artifacts with 2-3 dB of PSNR degradation in streaming scenarios. Second, while compression methods like quantization have been adopted to enable efficient streaming of 4DGS [5, 39], bitrate allocation across Gaussian attributes is typically handled in a uniform or heuristic manner [33, 42]. In contrast, we reveal that different Gaussian attributes exhibit highly heterogeneous sensitivity to quantization compression. Ignoring this disparity thus wastes significant bitrate, leading to a rate-distortion degradation of up to 6 dB PSNR in our analysis. More details for these two insights can be found in the motivation section.

To address these challenges, we present **PrismGS**, a coordinated training and compression framework for Pareto-optimal 4DGS streaming. First, we propose compression-aware **keyframe regularization** that explicitly accounts for the requirements of downstream dynamic training and compression (Sec. 3.1). Our approach constrains Gaussian scale and anisotropy during keyframe optimization instead of proceeding with naive static 3DGS training like prior methods. These regularization terms lead to compact and robust anchor representations and improve both the stability of dynamic propagation, as well as the resilience of Gaussian attributes to subsequent compression. Second, we develop an **attribute-level bitrate allocation** strategy for 4DGS streaming that explicitly exploit the heterogeneous bitrate-quality characteristics of different Gaussian attributes (Sec. 3.2). For each dynamic 3D sequence, PrismGS captures this property through a quality prediction model for attribute-level rate-distortion analysis. It then uses a dynamic programming algorithm to identify the Pareto frontier of the rate-quality trade-off. Our fine-grained design significantly improves compression efficiency and visual quality, while remaining compatible with standard video codecs. In addition, we introduce a **parallel decoding architecture** (Sec. 3.3) leveraging frame-wise independency in our compression, which boosts decoding speed to



(a) Error accumulation in training. (b) Visual artifacts after compression. (c) Various compression sensitivity.

Figure 1: Motivation study. (a) Vanilla keyframe reconstruction leads to error accumulation in training. (b) Large Gaussian primitives blur the background (left), highly-anisotropic Gaussian primitives damage object structure (right) under compression. (c) Different Gaussian attributes show distinct sensitivity to compression.

over 50 FPS and enables real-time decoding and rendering on the client side. In summary, our key contributions are as follows:

- We propose PrismGS, a coordinated training and compression framework for 4DGS streaming via our novel insights into keyframe construction and attribute-wise sensitivity to compression.
- We propose a key frame regularization strategy that controls Gaussian scale and anisotropy to improve dynamic propagation accuracy and compressibility.
- We introduce an attribute-level bitrate allocation mechanism leveraging different compression sensitivity, achieving Pareto-optimal rate-distortion performance through efficient DP algorithm under our quality prediction model.
- Extensive experiments on real-world datasets demonstrate PrismGS's superiority over existing methods in terms of fidelity, model size, and rendering, decoding speed.

2 Preliminary and Motivation

2.1 3DGS and Its Extension to Dynamic Scenes

3D Gaussian Splatting (3DGS) [15] represents a scene as a set of 3D Gaussian primitives G. Each primitive G_i is characterized by a set of attributes: $G_i = \{\boldsymbol{\mu}_i, \alpha_i, \mathbf{c}_i, \mathbf{R}_i, \mathbf{s}_i\}$, where $\boldsymbol{\mu}_i$ is its 3D center position, α_i its opacity, $\mathbf{c}_i$ its color, $\mathbf{R}_i$ and $\mathbf{s}_i$ are rotation and scaling matrices that define its 3D covariance $\boldsymbol{\Sigma}_i = \mathbf{R}_i\mathbf{s}_i\mathbf{s}_i^T\mathbf{R}_i^T$. The Gaussian kernel G_i is defined as $\exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^T\boldsymbol{\Sigma}_i^{-1}(\mathbf{x}-\boldsymbol{\mu}_i)\right)$, where $\mathbf{x}$ is any 3D point.

In rendering, these primitives are projected onto the 2D image plane via differentiable splatting. The color C of a pixel is computed by blending N ordered overlapping Gaussians, namely, $C = \sum_{i\in N}\mathbf{c}_i\alpha_i\prod_{j=1}^{i-1}(1-\alpha_j)$, where $\mathbf{c}_i$ and α_i are the projected color and opacity followed by the accumulated transmittance.

In training process, the parameters are optimized through a differentiable renderer supervised by

$$\mathcal{L}_{\text{rec}} = (1-\lambda_{\text{dssim}})\|\hat{I}-I\|_1 + \lambda_{\text{dssim}}\big(1-\text{SSIM}(\hat{I},I)\big), \qquad (1)$$

where $\hat{I}$ and I denote the rendered and ground-truth image, respectively, ℓ_1 loss measures pixel-wise error, $\text{SSIM}(\cdot,\cdot)$ denotes the structural similarity index, λ_{dssim} balances the contribution between the ℓ_1 term and the DSSIM term.

To efficiently extend 3DGS to dynamic scenes, most recent methods adopt a keyframe-residual training paradigm to reduce temporal redundancy [7, 29, 31]. In this paradigm, a static keyframe is first reconstructed using standard 3DGS optimization, and subsequent frames are obtained by modeling scene dynamics and inter-frame variations [19, 37]. The dynamic scenes can be displayed by sequentially rendering all frames.

2.2 Motivation

In this section, we propose two insights regarding current research gaps in 4DGS training and compression pipeline. We first train and compress frame sequence for cook_spinach scene in N3DV dataset [21] following V^3 [33] pipeline. We then compress each attribute in 3DGS representation using different quantization steps while maintaining others uncompressed and record the relative visual quality in PSNR.

Insight 1: Suboptimal keyframe primitives cause systemic R-D degradation across the 4DGS pipeline. Existing keyframe-residual training methods focus on dynamic motion modeling and often overlook the static keyframe's role, inheriting vanilla 3DGS optimization in Eq. 1. Prior works [18, 36] have revealed that **excessively large and highly anisotropic** Gaussian primitives degrade visual fidelity in static 3DGS reconstruction. In the context of 4DGS streaming, we conduct experiments to verify the effect of such primitives in training and compression:

First, in **dynamic training**, unconstrained keyframe Gaussians serve as poor anchors, leading to unstable residual propagation and rapid error accumulation over time, as shown in Fig. 1a. The visual quality first increases due to continuous refinement, then damaged by the error accumulation introduced by excessive large and anisotropic primitives.

Second, in **post-training compression**, large Gaussian scales and highly anisotropic shapes substantially amplify quantization noise. As illustrated in Fig. 1b, large Gaussians exhibit increased dynamic range and entropy, making them more susceptible to quantization errors that manifest as blurring in homogeneous regions like the wall in the background shown on the left; extreme anisotropy

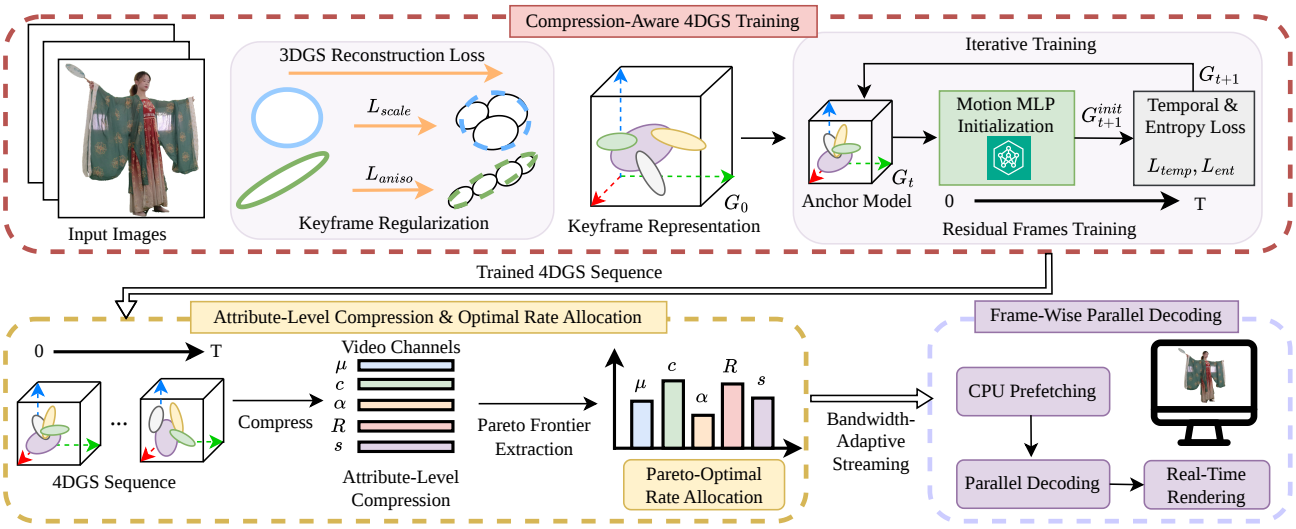


Figure 2: System design overview of the PrismGS framework. Our approach integrates a compression-aware 4DGS training pipeline to generate robust Gaussian sequences, which are then decoupled into attribute-level video channels. We apply a Pareto-optimal rate allocation strategy to optimize the rate-distortion trade-off for efficient streaming. Finally, the parallel decoding architecture leverages frame-wise independence to reduce decoding latency, facilitating real-time 4DGS rendering.

magnifies perturbations in rotation attributes, causing structural distortions in elongated objects such as the shelf on the right.

Insight 2: Gaussian attributes vary in compression sensitivity, creating new optimization opportunities for more efficient encoding. As shown in Fig. 1c, different Gaussian attributes exhibit varying sensitivity to compression. Attributes such as scale and rotation have a more pronounced impact on reconstruction quality under aggressive quantization compared to opacity. This uneven sensitivity suggests that a uniform compression strategy across attributes is suboptimal and that the encoding scheme can be further optimized.

3 System Design

Motivated by the novel insights we revealed in the motivation part, we present PrismGS, a coordinated training and compression pipeline for real-time 4DGS streaming. The overall design of PrismGS is shown in Fig. 2. It begins with regularized keyframe reconstruction, where we introduce scale and anisotropy regularization to penalize extreme Gaussian primitives to reduce dynamic training degradation and compression loss. We then adopt a prevalent residual-based dynamic training incorporating temporal coherence loss and entropy-aware regularization. Together, these components form our compression-aware training pipeline (Sec. 3.1). After training completes, an attribute-level compression scheme is applied to achieve optimized rate allocation across different Gaussian attributes, enabling Pareto-optimal rate-distortion performance (Sec. 3.2). Notably, the Pareto frontier consists of various optimal bitrate-quality points from which the system can select to achieve bandwidth-adaptive streaming. Finally, we introduce a parallel decoding architecture at the client side, which effectively reduces decoding latency and maintains synchronization with real-time rendering (Sec. 3.3).

3.1 Compression-Aware 4DGS Training

3.1.1 Keyframe Reconstruction with Regularization. To establish a robust foundation for subsequent dynamic training and improve compressibility, we introduce two regularization losses during keyframe reconstruction that penalize the generation of extremely large and anisotropic Gaussian primitives. As motivated by the analysis in Sec. 2.2, these Gaussian primitives lead to training instability and significant visual quality degradation after compression. Therefore, to overcome these shortcomings, two regularization terms are designed as follows.

Scale Regularization. To control the dynamic range of Gaussian primitives, we penalize large physical scales:

$$\mathcal{L}_{\text{scale}} = \mathbb{E}_i \left[\| \exp(\mathbf{s}_i) \|_2^2 \right]. \tag{2}$$

Here, $\mathbf{s}_i \in \mathbb{R}^3$ denotes the log-scale parameters of the i-th Gaussian, $\exp(\mathbf{s}_i)$ corresponds to the physical standard deviations along the principal axes, and $\| \cdot \|_2$ denotes the Euclidean norm. With the scale regularization term, excessively large Gaussian primitives are split into smaller ones, which are more robust to quantization and compression.

Anisotropy Regularization. We define the anisotropy ratio as the division of the largest and smallest scale of a Gaussian primitive:

$$\rho_i = \frac{\max(\exp(\mathbf{s}_i))}{\min(\exp(\mathbf{s}_i)) + \epsilon}, \tag{3}$$

where ϵ is a small constant to avoid numerical instability. Highly heterogeneous Gaussian primitives would cause unstable dynamic training and excessive visual quality loss after compression and quantization on rotation attribute. To reduce such loss, we penalize excessive anisotropy via

$$\mathcal{L}_{\text{aniso}} = \mathbb{E}_i \left[\max(0, \rho_i - \tau)^2 \right], \tag{4}$$

where ρ_i measures the degree of anisotropy of the i-th Gaussian, τ is a threshold that defines acceptable anisotropy necessary for scene reconstruction, under which the anisotropy is not penalized. In this way, Gaussian primitives with extreme anisotropy can be divided into several normal-shape ones.

Overall Objective. Absorbing these two loss terms, we have the full keyframe optimization objective combining reconstruction loss and regularization terms:

$$\mathcal{L}_{\text{keyframe}} = \mathcal{L}_{\text{rec}} + \lambda_s \mathcal{L}_{\text{scale}} + \lambda_a \mathcal{L}_{\text{aniso}}, \tag{5}$$

where $\mathcal{L}_{\text{rec}}$ is the reconstruction loss in Equation 1, λ_s and λ_a control the strength of the scale and anisotropy regularization, respectively. These regularizers increase the robustness of the keyframe for subsequent residual frames training. They also reduce the sensitivity of the rendering function to quantization noise and improve post-compression visual quality.

3.1.2 Residual-Based Dynamic Training. After we have the key frame being constructed, we proceed with subsequent dynamic training. the prevalent residual-based residual-based propagation strategy. Instead of independent per-frame optimization, we leverage the optimized Gaussians from the preceding frame G_t as an anchor to initialize the current frame G_{t+1}.

As illustrated in Fig. 2, given the optimized Gaussian representation of the previous frame G_t as an anchor, we utilize lightweight Multi-Layer Perceptrons (MLPs) to predict position attribute residuals and form the initialization of the next frame G_{t+1}^{init}. then undergoes a finetuning stage to capture frame details. To balance reconstruction fidelity with temporal stability and compression efficiency, we introduce the following regularizations:

Temporal Coherence ($\mathcal{L}_{\text{temp}}$), which penalizes abrupt attribute variations between G_t and G_{t+1} [33], enforcing a smooth trajectory in parameter space.

Entropy-Aware Regularization ($\mathcal{L}_{\text{ent}}$), which optimizes the residual distribution by minimizing the theoretical information entropy [1], encouraging a sparse representation.

Overall Objective. The full optimization objective for residual frames training stage is:

$$\mathcal{L}_{\text{dynamic}} = \mathcal{L}_{\text{rec}} + \lambda_{\text{t}} \mathcal{L}_{\text{temp}} + \lambda_{\text{e}} \mathcal{L}_{\text{ent}}, \tag{6}$$

where $\mathcal{L}_{\text{rec}}$ is the photometric loss in Eq. 1. λ_{t} and λ_{e} weight the temporal smoothness and entropy loss, respectively. The optimized G_{t+1} then becomes the anchor for the next frame, and the process proceeds iteratively across the sequence.

3.2 DP-based Optimal Bitrate Allocation for Efficient 4DGS Streaming

Efficient storage and streaming of 4D Gaussian Splatting (4DGS) representations requires compressing large numbers of per-frame Gaussian attributes under strict size constraints. In prior works, the problem of *optimal bitrate allocation across different Gaussian attributes* remains unexplored. Bitrate budgets are typically distributed uniformly across attributes [42]. However, as mentioned in Sec. 2.2, different Gaussian attributes vary in compression sensitivity, indicating that naive bitrate distribution among these attributes would lead to suboptimal RD performance.

To exploit attribute-wise rate-quality characteristics in practical settings, we adopt a compact video representation for 4DGS proposed in V^3 [33], in which each Gaussian attribute (except position) channel is independently normalized, quantized and encoded into 8-bit 2D video channels using a standard video codec (detailed in supplementary material). Position attribute is normalized into 16-bit and split into two 8-bit video channels. Each video channel can be compressed using various quantization parameters (qps), with larger qps lead to reduced size but increased compression loss (higher 8-bit video channels of position attribute are fixed to conduct lossless compression). This design provides a size-efficient representation for 4DGS and naturally supports existing codecs and streaming infrastructures, making it well suited for large-scale 4DGS content delivery.

Attribute-Level Configuration Space. To systematically study the optimal attribute-level rate allocation, we define an attribute-wise compression configuration as

$$\mathbf{qp} = (qp^1, qp^2, \ldots, qp^N), \quad q^i \in QP. \tag{7}$$

where the $\mathbf{qp}$ in bold represents a configuration, qp^i is the qp value of the ith Gaussian attribute, N is the number of Gaussian attributes, and QP is the set of feasible qp values. The total number of configurations is $|QP|^N$. Each configuration induces a per-frame size $R(\mathbf{qp})$ and rendering quality $Q(\mathbf{qp})$. $R(\mathbf{qp})$ is computed by adding the size of video channels for each attribute according to its qp value. To achieve optimal RD performance, our problem is to find configurations on Pareto frontier, where $R(\mathbf{qp})$ cannot be reduced without sacrificing $Q(\mathbf{qp})$.

Attribute-Wise Quality Modeling. Exhaustively evaluating all configurations is infeasible due to the exponential search space. Following previous study [39], we assume that the visual impacts of compressing different attributes are independent and model quality as the multiplication of all attributes.

$$Q(\mathbf{qp}) = Q_{\text{base}} \cdot \prod_{i=1}^{N_{\text{attr}}} f_i(qp^i), \tag{8}$$

where Q_{base} is the visual quality with $qp = 0$ for all attributes, $f_i(\cdot)$ denotes the normalized degradation function of attribute i. Each function is calibrated offline for each 4DGS sequence using the keyframe by varying the qp of a single attribute while fixing others to $qp = 0$.

Pareto Frontier Construction. We formulate attribute-level compression as a bi-objective optimization over bitrate and visual quality and construct the Pareto frontier [3], where bitrate consumption and quality cannot be further improved without sacrificing the other. Since exhaustive comparisons of large configuration space is infeasible, we design a dynamic programming searching algorithm to efficiently extract the Pareto frontier, as shown in Algorithm 1. It begins by initializing a dynamic programming table that tracks the best achievable quality at various bitrate levels. Starting from a base state of zero bitrate consumption and unit quality, the method iteratively considers each attribute in sequence. In the DP table, $dp[i][r]$ represents the best achieved visual quality for bitrate r considering first i attributes. For every attribute, the algorithm examines all previously computed states from the preceding stage

Algorithm 1 Pareto Frontier Extraction

Input: Attributes N, QP set Q, Bitrate $R_i(q)$, Quality $f_i(q)$, Budget $[r_{\min}, r_{\max}]$.
Output: Pareto frontier $\mathcal{P}$
Initialize $dp[N+1][r_{\max}/\Delta r] \leftarrow (-\infty, \emptyset)$
$dp[0][0] \leftarrow (1.0, \emptyset)$
for $i = 1$ **to** N **do**
  for $(r_{prev}, q_{prev}) \in dp[i-1]$ **do**
    for $q \in Q$ **do**
      $r_{new} \leftarrow r_{prev} + R_i(q), \quad q_{new} \leftarrow q_{prev} \times f_i(q)$
      $r_{idx} \leftarrow \lceil r_{new}/\Delta r \rceil$
      if $r_{new} \le r_{\max}$ **and** $q_{new} > dp[i][r_{idx}].quality$ **then**
        $dp[i][r_{idx}] \leftarrow (q_{new}, \text{path}_{prev} \cup \{q\})$
      end if
    end for
  end for
end for
return $\mathcal{P} = \{dp[N][r] \mid r \ge r_{\min}\}$

and extends them by evaluating every available quantization parameter. For each combination of previous state and quantization choice, the algorithm calculates the new cumulative bitrate and the corresponding quality impact. The bitrate is discretized using a fixed step size to maintain computational tractability. A new state is retained only if it remains within the total bitrate budget and improves upon the quality previously achievable at that bitrate level. The DP transition is defined as:

$$dp[i][r] = \max\{dp[i-1][r - R_i(qp^i)] \cdot f_i(qp^i)\}, \tag{9}$$

where $qp \in QP$, $r \ge R_i(qp^i)$, $R_i(qp^i)$ and $f_i^{\text{base}}(qp^i)$ denote the rate cost and relative quality contribution of attribute i under quantization parameter qp^i, respectively. The Pareto frontier can be found on the last column of the DP table with computational complexity $O\left(|N| \cdot |QP| \cdot \frac{r_{\max}}{\Delta r}\right)$, where $|N|$ for number of attributes, $|QP|$ for number of QP options, $r_{\max}$ for maximum bitrate and Δr for bitrate discretization step. The prediction error of the Pareto frontier is less than 0.25 dB with high Pearson coefficient on various scenes as shown in Tab. 1. The optimality proof and detailed complexity analysis are shown in supplementary material.

3.3 Frame-wise Parallel Decoding

While advances in 3DGS have achieved impressive rendering speeds, the decoding latency remains a critical yet often overlooked bottleneck for 4DGS streaming. Most existing dynamic 4DGS compression frameworks rely on either global optimization or sequential residual-chaining architectures [9, 19], imposing heavy inter-frame dependencies. Such serialized processing creates a significant decoding lag that prevents fluid, real-time interaction during streaming. In contrast, in PrismGS, by mapping Gaussian attributes into separate 2D video channels, we effectively break the temporal inter-frame constraints. In our framework, each frame is treated as a self-contained spatial-temporal unit. Correspondingly, we allow the decoder to bypass the sequential accumulation, and design a parallel decoding strategy leveraging such independence in our compression pipeline. Specifically, we adopt batched GPU operations that simultaneously decode multiple frames within a group. We further implement a background I/O worker to prefetch video channels and accelerate the waiting time. This architecture transforms the decoding process from a serialized bottleneck into a continuous stream, significantly masking I/O and computation latencies. As a result, PrismGS achieves high-throughput for decoding process that meets the real-time requirements of 4DGS content delivery.

Table 1: Prediction accuracy of the quality modeling module.

Dataset	PSNR MAE (dB) ↓	Pearson (r) ↑
HiFi4G [13]	0.18	0.9921
N3DV [21]	0.25	0.9932

4 Evaluation

4.1 Experiment Setup

Datasets. We evaluate our method and baselines using the HiFi4G Dataset [13] and the N3DV Dataset [21]. The HiFi4G Dataset consists of seven dynamic human movement sequences (simple scenes with large motion) and the N3DV Dataset includes six cooking scenes (complex scenes with smaller motion), both shot at 30 FPS.
Baselines. We compare PrismGS with following SOTA methods: 3DGStream [29], HiCoM [7], V^3 [33] and 4DGC [11], all of which are key-residual based end-to-end 4DGS training-compression method. While 3DGStream and HiCoM only have a single size option, V^3 and 4DGC employ rate control by adjusting training or compression parameters to support variable bitrate streaming.
Evaluation Metrics. We employ *Peak Signal-Noise Ratio (PSNR)* and *Structural Similarity Index Measure (SSIM)* to measure visual quality [10]. *Per-frame size (MB)* is used to evaluate storage and transmission efficiency. To assess real-time performance, we report both *rendering FPS* to evaluate the smoothness of the viewing experience and *decoding time per frame* to quantify the speed of decompressing 4DGS contents.
Implementation. All the training and evaluations are conducted using a 13th Gen Intel Core i7-13700 (2.10 GHz) with one NVIDIA GeForce RTX 3090 GPU. For keyframe reconstruction stage, $\lambda_{\text{dssim}} = 0.2$, $\lambda_s = \lambda_a = 3 \times 10^{-4}$. For dynamic training stage, $\lambda_{\text{temp}} = 1 \times 10^{-3}$, $\lambda_{\text{ent}} = 1 \times 10^{-4}$. Adam optimizer is used for all optimization process. More implementation details are in supplementary material.

4.2 Overall Performance

Quantitative Results. Tab. 2 summarizes the quantitative results on the HiFi4G [13] and N3DV [21] datasets. Our **PrismGS** consistently achieves SOTA performance across nearly all metrics. On the HiFi4G dataset, PrismGS reaches a PSNR of 34.60 dB and SSIM of 0.9841, outperforming the corresponding second-best method by 1.34 dB and 0.006, respectively, while significantly reducing the frame size by nearly 30% from 0.41 MB to only 0.3 MB compared to 4DGC. On the N3DV dataset, PrismGS maintains the leading PSNR of 31.66 dB and the smallest frame size of 0.67 MB. Notably, PrismGS achieves the highest rendering speeds for decoded models on both datasets, reaching 450 FPS and 295 FPS respectively, which is 1.16× and 1.08× faster than the second-best competitors. This gain stems from the regularized Gaussian representations, which

Table 2: Overall performance comparison on HiFi4G and N3DV datasets. Pink indicates the best performance in each metric, and Yellow indicates the second-best. PrismGS achieves Pareto-optimal rate-distortion performance across almost all metrics and significantly reduces decoding latency.

Method	HiFi4G Dataset [13]					N3DV Dataset [21]					Variable-Bitrate
	PSNR (dB)↑	SSIM↑	Size (MB)↓	Dec. Time (s)↓	FPS↑	PSNR (dB)↑	SSIM↑	Size (MB)↓	Dec. Time (s)↓	FPS↑	
3DGStream [29]	30.12	0.9671	7.7	-	250	30.15	0.9291	7.7	-	215	✗
HiCoM [7]	32.00	0.9708	0.60	-	370	31.17	0.9287	0.85	-	274	✗
V^3 [33]	33.26	0.9770	0.54	0.072	387	29.83	0.9100	0.95	0.085	260	✓
4DGC [11]	32.53	0.9779	0.41	0.080	210	31.64	**0.9365**	0.71	0.091	168	✓
PrismGS (Ours)	**34.60**	**0.9841**	**0.30**	**0.015**	**450**	**31.66**	0.9357	**0.67**	**0.021**	**295**	✓

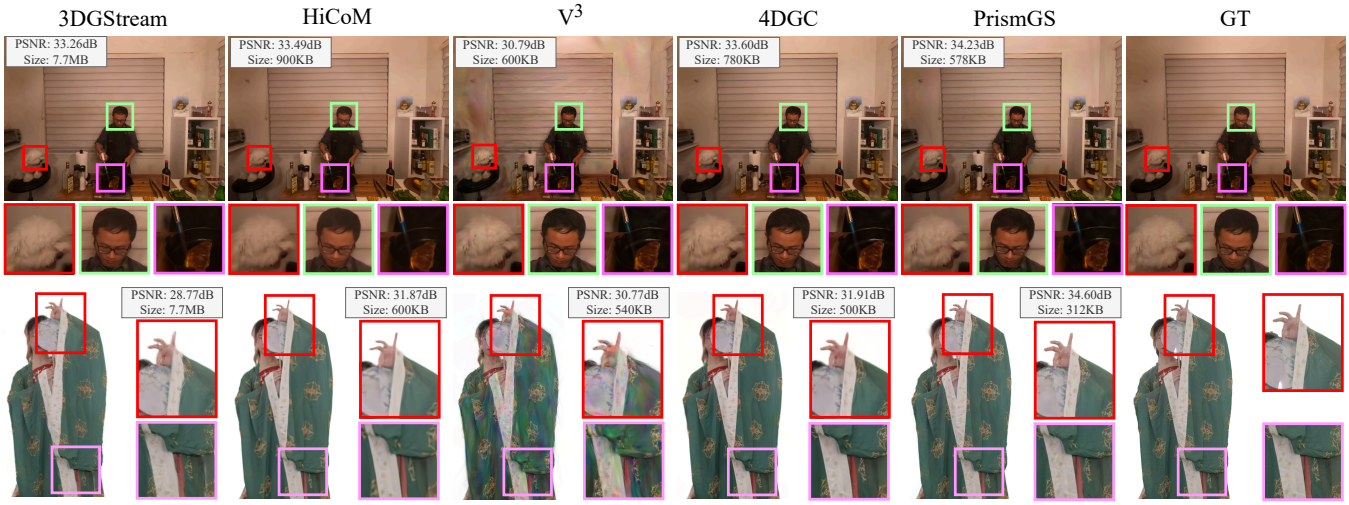


Figure 3: Qualitative comparison of PrismGS, baselines and ground truth (GT) in 4k_4_Dancing scene of HiFi4G dataset and flame_steak scene in N3DV dataset.

boost the rasterization efficiency. These results demonstrate that PrismGS successfully achieves a superior Pareto front, balancing high-fidelity reconstruction with competitive storage efficiency and real-time rendering capability.

In terms of decoding time, PrismGS exhibits a substantial lead in decoding efficiency. While methods like 4DGC and V^3 are constrained by heavy computational overhead in their sequential decompressors with only 10-15 FPS, PrismGS achieves a decoding speed of 50-60 FPS. This is attributed to the parallel decoding design, which allows for the simultaneous processing of independent attribute streams. This high decoding efficiency with smooth rendering speed ensure a end-to-end seamless and interactive 4D viewing experience. Per-scene quantitative results are listed in supplementary material.

Qualitative Comparison. We conduct a qualitative comparison with 3DGStream, V^3, HiCoM and 4DGC on the flame_steak scene in N3DV dataset and 4k_4_Dancing scene in HiFi4G dataset. As shown in Fig. 3, PrismGS achieves the lowest frame size while preserving the highest visual quality among all the methods. To be specific, PrismGS shows superiority in reconstructing finer details, such as the steak and face in flame_steak scene and the fan and hand in 4k_4_Dancing scene. This comparison further demonstrate PrismGS's ability in supporting smoother streaming and providing high-fidelity 4DGS contents.

RD Curve Analysis. To further evaluate the achieved quality in various bitrate conditions, we compare the rate-distortion (RD) curves of PrismGS and the baselines with variable bitrates (V^3 and 4DGC), as shown in Fig. 4. In RD curves, points located closer to the top-left indicate better performance—that is, higher visual quality with lower bitrate requirement at the same time and referred to as being dominant. In both datasets, PrismGS achieves the best quality-bitrate trade-off and consistently lies on the top-left of both the compared baselines, indicating the superiority of this coordinated 4DGS training and compression framework.

Overhead Analysis. Tab. 3 summarizes the per-stage overhead analysis of PrismGS on HiFi4G and N3DV datasets. Training begins with a 5-7 minute keyframe reconstruction, followed by per-frame prediction and optimization taking approximately one minute. For streaming usage, PrismGS first costs 2-3 minutes on Pareto frontier construction offline for each dynamic sequence, with both encoding and decoding latencies maintained under 0.1s per frame thanks to the parallelism support.

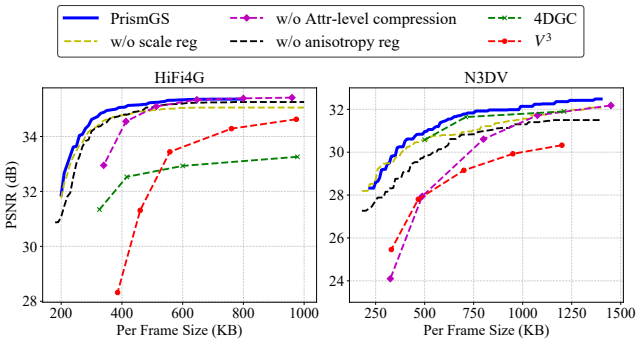


Figure 4: Rate-distortion curves on HiFi4G and N3DV dataset. PrismGS shows superiority across multiple baselines.

Table 3: System runtime and overhead analysis of PrismGS.

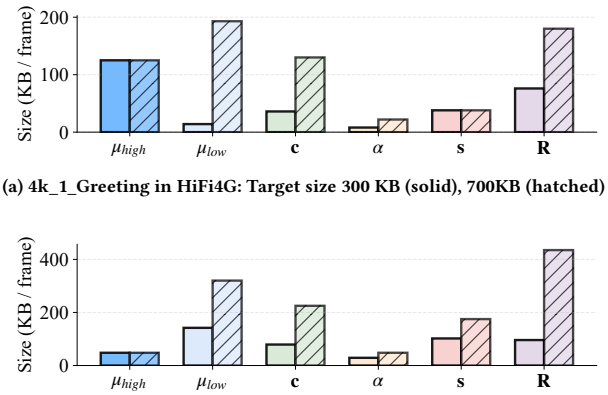
Phase	Mode	HiFi4G	N3DV
Training & Optimization			
Keyframe reconstruction	Offline	5 min	7 min
MLP prediction (per frame)	Offline	15 s	18 s
Finetuning (per frame)	Offline	40 s	53 s
Compression & Streaming			
Pareto frontier construction	Offline	2.6 min	3.3 min
Encoding (per frame)	Offline	0.04 s	0.06 s
Parallel Decoding (per frame)	**Online**	**0.015 s**	**0.021 s**

4.3 Anatomy of Rate Allocation in PrismGS

To further validate the effectiveness of our attribute-level compression module, we analyze the rate control and allocation behavior of PrismGS across different scenes and target bitrate budgets. Fig. 5 presents the detailed rate allocation under various size constraints. Notably, PrismGS dynamically allocates bitrate across attributes based on the target budget and their relative compression sensitivity. Under limited bitrate constraints, the model prioritizes preserving more compression-sensitive attributes—such as scale—to maintain basic visual quality. Conversely, when more bitrate is available, it distributes additional budget to less sensitive attributes to enhance fine details. Importantly, the sensitivity ranking varies across scenes, reflecting the ability of PrismGS to model attribute-level importance adaptively. For example, in the *coffee_martini* scene, μ_{low} is identified as more critical and thus receives a larger share of the bitrate under tight budgets to achieve optimized visual quality. This scene-dependent allocation demonstrates the capability of PrismGS to learn and exploit attribute-level sensitivity. Additional visualizations of rate control are in the supplementary material.

4.4 Ablation Study

We conduct ablation study on the contribution of each major component in our framework, including (i) scale regularization, (ii) anisotropy regularization, and (iii) attribute-level compression for Pareto-optimal rate allocation. (i) and (ii) are ablated by removing the corresponding loss term in keyframe training stage, while (iii) is ablated through replacing it with uniform QP assignment across all Gaussian attributes.



(a) 4k_1_Greeting in HiFi4G: Target size 300 KB (solid), 700KB (hatched)

(b) coffee_martini in N3DV: Target size 500 KB (solid), 1200KB (hatched)

Figure 5: Attribute-level rate allocation under different target sizes. Our PrismGS adaptively redistributes the rate budget, prioritizing sensitive attributes like μ_{low} and R as the total budget expands.

Table 4: Effect of keyframe regularization on training quality.

Dataset	PSNR (dB) ↑		SSIM ↑	
	w/o Reg.	w/ Reg.	w/o Reg.	w/ Reg.
HiFi4G	36.21	37.44 (+1.23)	0.9862	0.9890 (+0.028)
N3DV	33.01	34.27 (+1.26)	0.9636	0.9673 (+0.037)

RD Curve Analysis. We first provide a general view of the effect of these components across various size budgets using RD curves. As shown in Fig. 4, disabling scale regularization degrades performance across all size budgets on both datasets at 0.5-1 dB in PSNR. Removing anisotropy regularization term results in a more noticeable RD degradation of 0.7-1.2 dB in PSNR for both datasets. The effects validates the effectiveness of keyframe regularization in mitigating distortion amplification during compression. The attribute-level compression component exhibits outstanding performance in small-size region with PSNR improvement of up to 6 dB in N3DV dataset, since limited size budget make the need of optimal bitrate allocation more urgent and effective. This demonstrates the importance of fine-grained, attribute-aware bitrate allocation of PrismGS.

Effectiveness of Keyframe Regularization in Training. Tab. 4 presents the average PSNR and SSIM gains from scale and anisotropy regularization across HiFi4G and N3DV datasets. Regularization consistently improves reconstruction quality by over 1.2 dB PSNR and around 0.03 SSIM on average, demonstrating its effectiveness in stabilizing dynamic training and enhancing visual fidelity.

Qualitative Comparison. Fig. 6 presents rendered images with different components ablated. The full model achieves the best visual quality with the smallest size. Removing keyframe regularization introduces structural artifacts, particularly on fine details such as clothing folds and umbrella edges. Removing attribute-level compression severely degrades overall visual fidelity, resulting in noticeable blurring and distortion.

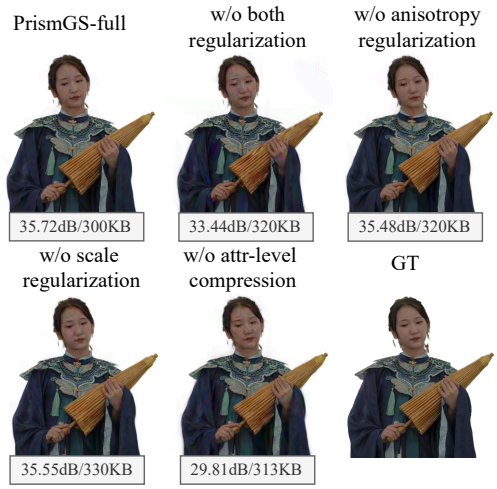


Figure 6: Qualitative comparison of ablation study on 4k_5_Umbrella scene in HiFi4G dataset.

Overall, the ablation results demonstrate that each component of our framework contributes meaningfully to training and compression robustness, with the strongest gains achieved when all components are jointly applied.

4.5 Sensitivity Analysis

To further investigate the influence of different weights of keyframe regularization, we examine and summarize the visual fidelity and model size in Tab. 5 with various weight configurations. Intensive regularization leads to bias in the initial reconstruction, reduced quality degradation after compression, and larger model size. The quantized visual quality first increases then decreases as the weights increase, while moderate level of regularization achieves superior post-compression quality. This analysis supports our selection of λ_{scale} and λ_{aniso} in the experiments.

5 Related Work

5.1 Dynamic 3D Scene Reconstruction

Early attempts at dynamic 3D scene reconstruction were primarily dominated by Neural Radiance Fields (NeRF) [24] extensions [2, 21, 26, 34, 47], which adopt temporal latent codes or deformation MLPs to represent time-varying geometries. However, these methods suffer from major bottlenecks of slow rendering speed and substantial per-frame storage overhead. The emergence of 3D Gaussian Splatting (3DGS) [15] revolutionized the field by introducing a point-based, explicit representation that allows for high-fidelity, real-time rendering. To extend this to the temporal domain, prior works [22, 37, 40, 41, 45] attempt to model the entire dynamic sequence globally by adding a temporal dimension to each Gaussian or utilizing 4D neural voxels. While these approaches achieve high reconstruction quality, they typically require prolonged training times and massive computational resources to converge on long sequences. Recent research [7, 9, 11, 29, 31, 43] has shifted toward keyframe-residual architectures, utilizing a canonical keyframe and employ deformation fields and MLPs to predict

Table 5: Impact of λ_{scale} and λ_{aniso} on compression and model size in N3DV: coffee_martini scene (Lossless: QP=0; Quantized: QP=20). Pink indicates the best performance.

	Lossless	Quantized		Model
λ	PSNR↑ (dB)	PSNR↑ (dB)	Degradation↓ (dB)	#Gaussians↓
0	**31.72**	26.16	5.56	**19.3k**
1×10^{-4}	31.67	28.32	3.35	19.6k
3×10^{-4}	31.48	**28.59**	2.89	19.8k
1×10^{-3}	31.09	28.40	**2.69**	20.2k

Gaussian displacements for subsequent residual frames. However, existing frameworks only adopt the original 3DGS initialization for keyframes without further adaptation, which ignores the different requirements and focus for static and dynamic scene reconstruction and leads to accumulated visual artifacts and quality degradation during long-term dynamic training. In contrast to the aforementioned methods, our proposed PrismGS introduces a novel keyframe regularization mechanism within a jointly-optimized framework, ensuring that the canonical representation of the keyframe provides a robust and stable foundation for temporal variations.

5.2 Dynamic 3D Scene Compression

Dynamic 3D scene compression is important for efficient streaming of large 3D scenes. The objective is to minimize the size required while preserving visual quality. NeRF-based VideoRF [32] and TeTriRF [38] apply tensor factorization and video codec for efficient compression. For 3DGS-based methods, an intuitive way is to prune Gaussian primitives with minor contribution based on their opacity [15] or other importance metrics [4, 12]. The variants of 3DGS leveraged anchor-based representation for compression [6, 23, 28, 30, 35] or adaptive quantization [39], which are computational intensive, especially for dynamic scenes. For dynamic scenes, exploiting temporal coherence has created another path. Keyframes are compressed with strong static compression, while intermediate frames encode motion and residuals relative to references [29, 44]. However, such methods suffer from compression degradation and long encoding-decoding time. V^3 [33] maps Gaussian attributes 2D image grids and then apply standard video codec (H.264) for compression with faster encoding-decoding speed, yet suboptimal bitrate allocation degrades its performance. In contrast, PrismGS achieves superior RD performance after compression through Pareto-optimal attribute-level bitrate allocation.

6 Conclusion

We propose PrismGS, a coordinated training and compression framework for 4DGS streaming. By integrating structural keyframe regularization with entropy-aware residual training, PrismGS establishes a robust, quantization-friendly foundation that ensures temporal smoothness and compressibility. Furthermore, we introduce an attribute-level quantization strategy and a quality prediction model to identify the Pareto-optimal rate-distortion trade-offs. Combined with a high-throughput parallel decoding architecture, PrismGS achieves superior visual quality and reduced data volume on HiFi4G and N3DV datasets, providing a practical, real-time solution for 4DGS content delivery.

References

[1] Jean Bégaint, Fabien Racapé, Simon Feltman, and Akshay Pushparaja. 2020. CompressAI: a PyTorch library and evaluation platform for end-to-end compression research. *CoRR* abs/2011.03029 (2020). arXiv:2011.03029 https://arxiv.org/abs/2011.03029

[2] Ang Cao and Justin Johnson. 2023. HexPlane: A Fast Representation for Dynamic Scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 130–141.

[3] Yair Censor. 1977. Pareto optimality in multiobjective problems. *Applied Mathematics and Optimization* 4, 1 (1977), 41–59.

[4] Yihang Chen, Qianyi Wu, Weiyao Lin, Mehrtash Harandi, and Jianfei Cai. 2025. HAC: Hash-Grid Assisted Context for 3D Gaussian Splatting Compression. In *Computer Vision – ECCV 2024*. Springer Nature Switzerland, Cham, 422–438.

[5] Zicong Chen, Zhenghao Chen, Wei Jiang, Wei Wang, Lei Liu, and Dong Xu. 2025. 4DGS-CC: A Contextual Coding Framework for 4D Gaussian Splatting Data Compression. *arXiv preprint arXiv:2504.18925* (2025).

[6] Zhiwen Fan, Kevin Wang, Kairun Wen, Zehao Zhu, Dejia Xu, and Zhangyang Wang. 2024. LightGaussian: Unbounded 3D Gaussian Compression with 15x Reduction and 200+ FPS. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[7] Qiankun Gao, Jiarui Meng, Chengxiang Wen, Jie Chen, and Jian Zhang. 2024. Hicom: Hierarchical coherent motion for dynamic streamable scenes with 3d gaussian splatting. *Advances in Neural Information Processing Systems (NeurIPS)* (2024).

[8] Georg Hess, Carl Lindström, Maryam Fatemi, Christoffer Petersson, and Lennart Svensson. 2025. SplatAD: Real-Time Lidar and Camera Rendering with 3D Gaussian Splatting for Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 11982–11992.

[9] Cheng-Yuan Ho, He-Bi Yang, Jui-Chiu Chiang, Yu-Lun Liu, and Wen-Hsiao Peng. 2026. TED-4DGS: Temporally Activated and Embedding-based Deformation for 4DGS Compression. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. 6371–6380.

[10] Alain Hore and Djemel Ziou. 2010. Image quality metrics: PSNR vs. SSIM. In *2010 20th international conference on pattern recognition*. IEEE, 2366–2369.

[11] Qiang Hu, Zihan Zheng, Houqiang Zhong, Sihua Fu, Li Song, Xiaoyun Zhang, Guangtao Zhai, and Yanfeng Wang. 2025. 4DGC: Rate-Aware 4D Gaussian Compression for Efficient Streamable Free-Viewpoint Video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 875–885.

[12] Yang Hu, Jingui Ma, Yucheng Yang, Jie Liang, Jinbo Yan, Jiahao Wu, Jiayu Yang, Yang Deng, and Ronggang Wang. 2025. Excavating the Most Critical Gaussians: Sparse Selection and Structural Optimization for Efficient 3DGS Compression. In *Proceedings of the 33rd ACM International Conference on Multimedia* (Dublin, Ireland) *(MM '25)*. Association for Computing Machinery, New York, NY, USA, 6967–6976.

[13] Yuheng Jiang, Zhehao Shen, Penghao Wang, Zhuo Su, Yu Hong, Yingliang Zhang, Jingyi Yu, and Lan Xu. 2024. Hifi4g: High-fidelity human performance rendering via compact gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. 19734–19745.

[14] Zhihui Ke, Xiaobo Zhou, Yuyang Liu, Zhizhuo Pang, and Tie Qiu. 2025. Edge-Gaussian: Real-time Free-Viewpoint Video for Mobile VR via Edge-Client Collaborative Neural Rendering. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 773–787.

[15] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.* 42, 4 (2023), 139–1.

[16] Hyunjeong Kim and In-Kwon Lee. 2024. Is 3DGS Useful?: Comparing the Effectiveness of Recent Reconstruction Methods in VR. In *2024 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. 71–80.

[17] Jonghyun Kim, Michael Stengel, Amrita Mazumdar, Tianye Li, Cheng Sun, David Luebke, and Shalini De Mello. 2025. Play4D: Accelerated and Interactive Free-viewpoint Video Streaming for Virtual Reality and Light Field Displays. In *Proceedings of the SIGGRAPH Asia 2025 Emerging Technologies*. 1–3.

[18] Jeong Uk Lee and Sunghee Hee Choi. 2026. ARGS: Advanced Regularization on Aligning Gaussians over the Surface. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) Workshops*. 1298–1306.

[19] Deqi Li, Shi-Sheng Huang, Zhiyuan Lu, Xinran Duan, and Hua Huang. 2024. St-4dgs: Spatial-temporally consistent 4d gaussian splatting for efficient dynamic scene rendering. In *ACM SIGGRAPH 2024 Conference Papers*. 1–11.

[20] Ke Li, Reinhard Bacher, Susanne Schmidt, Wim Leemans, and Frank Steinicke. 2024. Reality Fusion: Robust Real-time Immersive Mobile Robot Teleoperation with Volumetric Visual Data Fusion. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 8982–8989.

[21] Tianye Li, Mira Slavcheva, Michael Zollhoefer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, Steven Lovegrove, Michael Goesele, Richard Newcombe, et al. 2022. Neural 3d video synthesis from multi-view video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. 5521–5531.

[22] Zhan Li, Zhang Chen, Zhong Li, and Yi Xu. 2024. Spacetime Gaussian Feature Splatting for Real-Time Dynamic View Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 8508–8520.

[23] Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. 2024. Scaffold-GS: Structured 3D Gaussians for View-Adaptive Rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 20654–20664.

[24] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2021. NeRF: representing scenes as neural radiance fields for view synthesis. *Commun. ACM* 65, 1 (Dec. 2021), 99–106.

[25] Zipeng Pan, Yuan Zhang, and Tao Lin. 2025. TeleGS: End-to-End Monocular Gaussian Head for Immersive Telepresence. In *Proceedings of the 3rd Workshop on Emerging Multimedia Systems* (Coimbra, Portugal) *(EMS '25)*. Association for Computing Machinery, New York, NY, USA, 37–42.

[26] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. 2021. D-nerf: Neural radiance fields for dynamic scenes. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. 10318–10327.

[27] Rui Song, Chenwei Liang, Yan Xia, Walter Zimmer, Hu Cao, Holger Caesar, Andreas Festag, and Alois Knoll. 2025. Coda-4dgs: Dynamic gaussian splatting with context and deformation awareness for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 28031–28041.

[28] Zetian Song, Jiaye Fu, Jiaqi Zhang, Xiaohan Lu, Chuanmin Jia, Siwei Ma, and Wen Gao. 2026. TinySplat: Feedforward Approach for Generating Compact 3D Scene Representation. *IEEE Transactions on Circuits and Systems for Video Technology* (2026), 1–1.

[29] Jiakai Sun, Han Jiao, Guangyuan Li, Zhanjie Zhang, Lei Zhao, and Wei Xing. 2024. 3dgstream: On-the-fly training of 3d gaussians for efficient streaming of photo-realistic free-viewpoint videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 20675–20685.

[30] Xiangyu Sun, Joo Chan Lee, Daniel Rho, Jong Hwan Ko, Usman Ali, and Eunbyung Park. 2024. F-3DGS: Factorized Coordinates and Representations for 3D Gaussian Splatting. In *Proceedings of the 32nd ACM International Conference on Multimedia* (Melbourne VIC, Australia) *(MM '24)*. Association for Computing Machinery, New York, NY, USA, 7957–7965.

[31] Diwen Wan, Ruijie Lu, and Gang Zeng. 2024. Superpoint Gaussian Splatting for Real-Time High-Fidelity Dynamic Scene Reconstruction. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 49957–49972.

[32] Liao Wang, Kaixin Yao, Chengcheng Guo, Zhirui Zhang, Qiang Hu, Jingyi Yu, Lan Xu, and Minye Wu. 2024. VideoRF: Rendering Dynamic Radiance Fields as 2D Feature Video Streams. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 470–481.

[33] Penghao Wang, Zhirui Zhang, Liao Wang, Kaixin Yao, Siyuan Xie, Jingyi Yu, Minye Wu, and Lan Xu. 2024. V^ 3: Viewing Volumetric Videos on Mobiles via Streamable 2D Dynamic Gaussians. *ACM Transactions on Graphics (TOG)* 43, 6 (2024), 1–13.

[34] Yiming Wang, Qin Han, Marc Habermann, Kostas Daniilidis, Christian Theobalt, and Lingjie Liu. 2023. NeuS2: Fast Learning of Neural Implicit Surfaces for Multi-view Reconstruction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 3295–3306.

[35] Yufei Wang, Zhihao Li, Lanqing Guo, Wenhan Yang, Alex C. Kot, and Bihan Wen. 2024. ContextGS : Compact 3D Gaussian Splatting with Anchor Level Context Model. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[36] Yongxiang Wang and Gang Zhou. 2026. SPDGS: Spatial Pruning and Depth Priors for Sparse-View 3D Gaussian Splatting. In *MultiMedia Modeling*. Springer Nature Singapore, Singapore, 46–59.

[37] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 2024. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 20310–20320.

[38] Minye Wu, Zehao Wang, Georgios Kouros, and Tinne Tuytelaars. 2024. TeTriRF: Temporal Tri-Plane Radiance Fields for Efficient Free-Viewpoint Video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 6487–6496.

[39] Shuzhao Xie, Jiahang Liu, Weixiang Zhang, Shijia Ge, Sicheng Pan, Chen Tang, Yunpeng Bai, Cong Zhang, Xiaoyi Fan, and Zhi Wang. 2025. SizeGS: Size-aware Compression of 3D Gaussian Splatting via Mixed Integer Programming. In *Proceedings of the 33rd ACM International Conference on Multimedia* (Dublin, Ireland) *(MM '25)*. Association for Computing Machinery, New York, NY, USA, 8214–8223.

[40] Jiawei Xu, Zexin Fan, Jian Yang, and Jin Xie. 2024. Grid4D: 4D Decomposed Hash Encoding for High-Fidelity Dynamic Gaussian Splatting. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[41] Jinbo Yan, Rui Peng, Luyang Tang, and Ronggang Wang. 2024. 4D Gaussian Splatting with Scale-aware Residual Field and Adaptive Optimization for Real-time Rendering of Temporally Complex Dynamic Scenes. In *Proceedings of the*

32nd ACM International Conference on Multimedia (Melbourne VIC, Australia) *(MM '24)*. Association for Computing Machinery, New York, NY, USA, 7871–7880.

[42] Qi Yang, Le Yang, Geert Van Der Auwera, and Zhu Li. 2025. HybridGS: High-Efficiency Gaussian Splatting Data Compression using Dual-Channel Sparse Representation and Point Cloud Encoder. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, Vancouver, Canada.

[43] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. 2024. Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 20331–20341.

[44] Wenkang Zhang, Yan Zhao, Qiang Wang, Li Song, and Zhengxue Cheng. 2025. D-FCGS: Feedforward Compression of Dynamic Gaussian Splatting for Free-Viewpoint Videos. *arXiv preprint arXiv:2507.05859* (2025).

[45] Xinjie Zhang, Zhening Liu, Yifan Zhang, Xingtong Ge, Dailan He, Tongda Xu, Yan Wang, Zehong Lin, Shuicheng Yan, and Jun Zhang. 2025. MEGA: Memory-Efficient 4D Gaussian Splatting for Dynamic Scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 27828–27838.

[46] Guosheng Zhao, Chaojun Ni, Xiaofeng Wang, Zheng Zhu, Xueyang Zhang, Yida Wang, Guan Huang, Xinze Chen, Boyuan Wang, Youyi Zhang, Wenjun Mei, and Xingang Wang. 2025. DriveDreamer4D: World Models Are Effective Data Machines for 4D Driving Scene Representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 12015–12026.

[47] Zihan Zheng, Houqiang Zhong, Qiang Hu, Xiaoyun Zhang, Li Song, Ya Zhang, and Yanfeng Wang. 2024. HPC: Hierarchical Progressive Coding Framework for Volumetric Video. In *Proceedings of the 32nd ACM International Conference on Multimedia* (Melbourne VIC, Australia) *(MM '24)*. Association for Computing Machinery, New York, NY, USA, 7937–7946.